

▼ Aula de Strings em Python

Introdução

Em Python, **textos, palavras e até frases inteiras** são representados pelo tipo de dado `str`, conhecido como *string* (cadeia de caracteres).

Diferente de linguagens como C ou C++, o Python **não possui o tipo `char`**, mesmo um único caractere é tratado como uma string de tamanho 1.

Uma *string* é uma **sequência imutável de caracteres**, usada para representar textos dentro de um programa.

Elas podem ser criadas de várias formas, usando aspas simples `'...'`, aspas duplas `"..."`, ou até aspas triplas `'''...'''` ou `"""..."""` para textos longos ou multilinhas.

Strings são um dos tipos de dados **mais importantes e versáteis** do Python, sendo utilizadas para:

- Exibir mensagens para o usuário 🗣️
- Manipular nomes, endereços, ou informações textuais 📄
- Ler e processar arquivos 📁
- Montar respostas automáticas, relatórios e mensagens formatadas 📧

Nesta aula, você vai aprender:

1. Como **criar e exibir** strings no Python.
2. Como usar **caracteres especiais** e *escape sequences*.
3. Como **acessar partes de uma string** com *index* e *slicing*.
4. Como **juntar ou repetir** strings com operadores.
5. Como **formatar textos dinamicamente**, usando `format()` e **f-strings**.
6. E conhecer **funções** mais comuns de strings.

Criando uma String

Para começar, vamos ver como criar uma string em python:s

```
1 # Aspas simples e duplas para texto "in line"
2 nome = "João Pedro"
3 sobrenome = 'B. Garcia'
4 print(nome)
5 print(sobrenome)
```

```
João Pedro
B. Garcia
```

```
1 # Aspas triplas para texto multilinhas
2 nome_completo = """João Pedro
3 B. Garcia
4 """
5 print(nome_completo)
```

```
João Pedro
B. Garcia
```

Note que, ao mudar de linha dentro do código, a **string também muda de linha** quando é exibida no terminal.

No entanto, **não é necessário usar aspas triplas** toda vez que quisermos quebrar uma linha em um texto.

Podemos usar o **caractere especial** `\n`, conhecido como *newline* (*nova linha*), que indica ao Python que o texto a seguir deve ser exibido na próxima linha.

Veja o exemplo abaixo:

```
1 texto_com_quebra_de_linha = "Texto\ncom\nquebra\nde\nlinha"
2 print(texto_com_quebra_de_linha)
```

```
Texto
com
quebra
```

de
linha

De forma geral, a **barra invertida** (`\`) é usada para inserir **caracteres especiais** dentro de uma string. Ela serve como um sinal para o Python entender que o próximo símbolo deve ser interpretado de forma diferente.

Por exemplo, podemos usá-la para **incluir aspas dentro de uma string** sem causar erro na interpretação do código:

```
1 texto_com_aspas = "Texto \"com\" aspas"
2 print(texto_com_aspas)
```

Texto "com" aspas

Existem muitos caracteres especiais, veja a tabela abaixo como exemplo:

Escape	Significado	Exemplo	Resultado
<code>\\</code>	Barra invertida literal	<code>"C:\\Users\\Joao"</code>	<code>C:\\Users\\Joao</code>
<code>\'</code>	Aspas simples	<code>'It's ok'</code>	<code>It's ok</code>
<code>\"</code>	Aspas duplas	<code>"Ele disse: \"Oi\""</code>	<code>Ele disse: "Oi"</code>
<code>\n</code>	Nova linha (line break)	<code>"Olá\nMundo"</code>	<code>Olá Mundo</code>
<code>\t</code>	Tabulação (tab)	<code>"A\tB\tC"</code>	<code>A B C</code>
<code>\r</code>	Retorno de carro (carriage return)	<code>"ABC\rD"</code>	<code>DBC</code>
<code>\b</code>	Backspace (apaga o caractere anterior)	<code>"ABC\bD"</code>	<code>ABD</code>
<code>\f</code>	Form feed (avanço de página, pouco usado)	<code>"A\fB"</code>	depende do terminal
<code>\v</code>	Tabulação vertical (vertical tab)	<code>"A\vB"</code>	depende do terminal
<code>\a</code>	Alerta sonoro (beep, depende do sistema)	<code>"\a"</code>	🔔 (emite som)
<code>\uXXXX</code>	Caractere Unicode (4 dígitos hexadecimais)	<code>"\u00E1"</code>	á
<code>\UXXXXXXXX</code>	Caractere Unicode (8 dígitos)	<code>"\U0001F600"</code>	😄
<code>\N{NOME}</code>	Caractere Unicode por nome	<code>"\N{LATIN SMALL LETTER A WITH ACUTE}"</code>	á
<code>\xhh</code>	Caractere ASCII por código hexadecimal	<code>"\x41"</code>	A
<code>\ooo</code>	Caractere ASCII por código octal	<code>"\101"</code>	A

▼ Acessando Strings

Em Python, podemos acessar **letras específicas** de uma string ou **recortar partes** dela para analisar apenas um trecho. Essas duas operações são feitas com o que chamamos de **indexação** e **slicing**.

Acessando por índice (*index*)

Cada caractere dentro de uma string tem uma **posição numérica**, chamada de *índice*.

Os índices começam em **0** e aumentam da **esquerda para a direita**.

Por exemplo:

```
letra = texto[0] # Acessa o primeiro caractere da string
```

Fatiando strings (slicing)

O slicing é uma forma de **recortar** partes da string, indicando o índice inicial e o índice final desejados. A fatia começa no primeiro índice informado e **vai até o anterior ao índice final**.

```
palavra = texto[5:17]
```

Esse comando seleciona todos os caracteres entre as posições 5 e 16 do texto original.

```
1 # Acessando por Índice:
2 # Considere esse texto a seguir:
3 texto = "Mobile Robotics Group, é o laboratório de robôs móveis da USP São Carlos"
4
5 # Queremos pegar a primeira letra dessa string, o "M". Podemos fazer isso:
6 letra = texto[0]
7 print(letra)
```

M

```
1 # Como comentado a string é imutavel, parecida com a Tupla.
2 # Então não conseguimos alterar as letras usando o index
3 texto[0] = "m"
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-4282094423.py in <cell line: 0>()
    1 # Como comentado a string é imutavel, parecida com a Tupla.
    2 # Então não conseguimos alterar as letras usando o index
----> 3 texto[0] = "m"

TypeError: 'str' object does not support item assignment
```

✓ Slicing

Se você contar as letras, o nome do laboratório começa na primeira letra (index 0) e termina no index 20. Podemos selecionar essa parte fazendo um slicing na string, selecionando os caracteres de 0 a 20.

Como comentado quando fazemos o slicing, ele começa na letra do primeiro index e vai até o anterior do último index. Por isso nosso slicing tem que ser:

```
1 nome_lab = texto[0:21]
2 print(nome_lab)
```

Mobile Robotics Group

```
1 # No caso em específico, que o slicing começa no zero,
2 # podemos ocultar ele. Assim temos:
3 nome_lab = texto[:21]
4 print(nome_lab)
```

Mobile Robotics Group

Em linguagens como **C++** (e várias outras), não existem **índices negativos**.

Mas pare pra pensar um pouco... 🤔

Se eles existissem, o que você acha que fariam?

No **Python**, isso realmente existe — e é um recurso **muito útil!**

Os **índices negativos** permitem acessar elementos **a partir do final** da string, em vez do início.

Por exemplo:

```
texto = "Python"
print(texto[-1]) # Última letra
print(texto[-3]) # Terceira letra a partir do final
```

Isso é especialmente prático quando queremos referenciar o final da string sem precisar saber seu tamanho.

```
1 # pegando a 10 letra a partir do Final ("S" de "São Carlos")
2 print(texto[-10])
```

S

```
1 # Funciona também com Slicing
2 print(texto[-30:-10])
```

robôs móveis da USP

```
1 # E também podemos omitir o 0, quando queremos pegar a partir do final da string
2 # Nesse caso o -10 fica antes do ":", já que -10 é menor que 0
3 print(texto[-10:])
```

São Carlos

✓ Formatação de Strings

Em programação, é muito comum precisarmos **exibir informações de forma organizada e legível**.

Por exemplo, mostrar valores numéricos dentro de frases, alinhar textos ou controlar o número de casas decimais em um resultado.

Em **Python**, tudo isso é feito por meio da **formatação de strings**, um recurso essencial para criar **saídas mais claras, dinâmicas e profissionais**.

Com a formatação, podemos combinar textos com variáveis, números e resultados de cálculos, mantendo tudo bonito e bem organizado na tela.

Vamos ver como isso funciona na prática 📌

✓ Concatenação

A **concatenação** é o processo de **juntar duas ou mais strings** para formar uma única.

Em Python, isso é feito de forma simples, usando o **operador** `+`.

Por exemplo:

```
1 nome = "João Pedro"
2 sobrenome = "B. Garcia"
3 nome_completo = nome + " " + sobrenome
4 print(nome_completo)
```

João Pedro B. Garcia

```
1 # Também funciona de forma incremental
2 saudacao = "Olá"
3 bom_dia = ", bom dia!"
4 saudacao += bom_dia
5 print(saudacao)
```

Olá, bom dia!

Podemos também, repetir varias vezes uma string utilizando o operador `*`:

```
1 risada = "Ha"
2 muita_risada = risada * 3
3 print(muita_risada)
```

HaHaHa

Uma forma de adicionar resultados em uma string, é utilizando a conversão de tipos. Podemos utilizar o método `str()` para transformar uma variavel numérica em texto.

```
1 numero = str(10)
2 print("Número: " + numero)
3 print(type(numero))
```

Número: 10
<class 'str'>

▼ Método `.format()` e F-strings — formatação inteligente de texto

O Python oferece formas muito práticas e poderosas de **inserir valores dentro de strings** sem precisar usar o operador `+`. Essas formas tornam o código **mais limpo, legível e fácil de manter**, especialmente quando trabalhamos com várias variáveis.

As duas principais maneiras são:

- O método `.format()`, disponível desde versões antigas do Python.
- As **F-strings** (*Format String Literals*), introduzidas no Python 3.6, que são mais modernas e diretas.

Ambas permitem **misturar texto e variáveis** dentro da mesma string, controlando inclusive a **formatação de números, casas decimais e alinhamento de texto**.

Vamos conhecer cada uma delas:

O método `.format()` foi, por muito tempo, a principal forma de **inserir variáveis dentro de uma string** em Python. Ele funciona substituindo **marcadores** `{}` dentro do texto pelos valores passados como argumento.

```
1 # metodo .format()
2 nome = "João"
3 idade = 22
4 print("Meu nome é {} e tenho {} anos.".format(nome, idade))
```

```
Meu nome é João e tenho 22 anos.
```

Os colchetes `{}` são substituídos pelos valores passados para `.format()` na ordem em que aparecem. Mas podemos indicar quais argumentos queremos usar em cada posição, escrevendo o índice dentro das chaves e até argumentos nomeados:

```
1 # Indices e Nomeados
2 print("0 {0} custa {1} reais. E acabei comprando o {0}.".format("livro", 30))
3 print("0 {produto} custa {preco} reais. E acabei comprando o {produto}.".format(produto="livro", preco=30))
```

```
0 livro custa 30 reais. E acabei comprando o livro
0 livro custa 30 reais. E acabei comprando o livro
```

Podemos também formatar melhor os números:

```
1 pi = 3.14159265359
2 print("Sou Físico, e utilizo pi = {}".format(pi))
3 print("Sou Engenheiro, e utilizo pi = {:.2f}".format(pi))
```

```
Sou Físico, e utilizo pi = 3.14159265359
Sou Engenheiro, e utilizo pi = 3.14
```

Outra função poderossíssima é o alinhamento e espaçamento de strings. Veja o exemplo a baixo e depois vamos destinar em como fazer ele:

```
1 print("{:<10} | {:>8}".format("Produto", "Preço"))
2 print("{:<10} | {:>8.2f}".format("Livro", 30))
3 print("{:<10} | {:>8.2f}".format("Caneta", 2.5))
```

```
Produto | Preço
Livro   | 30.00
Caneta  | 2.50
```

Podemos reservar um **espaço mínimo** para cada valor dentro da string.

Basta indicar o número de caracteres desejado após o símbolo `:`.

```
print("{:10}".format("Python"))
>> Python
```

Neste exemplo, a palavra "Python" ocupa 10 espaços, sendo preenchida com espaços vazios à direita (por padrão, alinhamento à esquerda).

```
1 nome1 = "João Pedro"
2 nome2 = "Maria"
```

```
3 universidade = "USP"
4 print("{:10} estuda na {:10}".format(nome1, universidade))
5 print("{:10} estuda na {:10}".format(nome2, universidade))
```

```
João Pedro estuda na USP
Maria          estuda na USP
```

E por ultimo podemos controlar o alinhamento:

Símbolo	Alinhamento	Exemplo
<	Esquerda	"{:<10}".format("A")
>	Direita	"{:>10}".format("A")
^	Centro	"{: ^10}".format("A")

```
1 nome1 = "João Pedro"
2 nome2 = "Maria"
3 universidade = "USP"
4 print("{:>10} estuda na {:10}".format(nome1, universidade))
5 print("{:>10} estuda na {:10}".format(nome2, universidade))
```

```
João Pedro estuda na USP
Maria estuda na USP
```

Como vocês viram, podemos combinar varios formatações:

```
1 print("{:<10} | {:>8}".format("Produto", "Preço"))
2 print("{:<10} | {:>8.2f}".format("Livro", 30))
3 print("{:<10} | {:>8.2f}".format("Caneta", 2.5))
```

```
Produto | Preço
Livro   | 30.00
Caneta  | 2.50
```

Nesse caso, repare que em `{:>8.2f}`. Estamos alinhando a esquerda, deixando 8 espaços e colocando 2 casas decimais para o número.

As **f-strings** permitem fazer a mesma formatação do método `.format()`, mas de forma **muito mais simples e direta**.

Basta colocar a variável dentro das chaves `{}` e, após o `:`, usar os mesmos especificadores de **largura, alinhamento e preenchimento**.

```
nome = "Python"
print(f"{nome:10}")
>> Python
```

```
1 produto = "Livro"
2 preco = 30
3 print(f"{produto:<10} | {preco:>8.2f}")
```

```
Livro | 30.00
```

```
1 print(f"{'A':-^10}") # centralizado e preenchido com '-'
2 print(f"{'B':.>10}") # alinhado à direita e preenchido com '.'
3 print(f"{'C':~<10}") # alinhado à esquerda e preenchido com '~')
```

```
----A-----
.....B
C~~~~~
```

Como as f-strings permitem interpolar expressões diretamente, podemos até formatar resultados de operações:

```
1 valor = 123.4567
2 print(f"0 valor arredondado é {valor:>10.2f}")
3 print(f"A metade do valor é {(valor / 2):^10.3f}")
```

```
0 valor arredondado é 123.46
A metade do valor é 61.728
```

Buscas em Strings

Quando trabalhamos com textos, é muito comum precisarmos **verificar ou localizar informações específicas** dentro de uma string. Por exemplo:

- Verificar se uma frase **começa** ou **termina** com uma certa palavra.
- Procurar se uma **substring** (parte de um texto) está presente dentro de outra.
- Contar quantas vezes um termo aparece.

O Python oferece vários métodos simples e poderosos para fazer esse tipo de busca e verificação.

Essas ferramentas são essenciais quando lidamos com **análise de textos**, **validação de dados** ou **processamento de informações de entrada**.

Vamos conhecer as principais formas de trabalhar com buscas em strings:

```
1 # "in" e "not in"
2 texto = "Robótica Móvel na USP São Carlos"
3
4 print("USP" in texto)
5 print("UFSCar" not in texto)
```

```
True
True
```

```
1 # "startswith" e "endswith"
2 frase = "Aprendendo Python para robótica"
3
4 print(frase.startswith("Aprendendo"))
5 print(frase.endswith("robótica"))
6 print(frase.endswith("C++"))
```

```
True
True
False
```

```
1 # O método find() retorna a posição (índice) onde a substring aparece pela primeira vez.
2 # Se ela não for encontrada, o resultado é -1.
3 texto = "Mobile Robotics Group - USP São Carlos"
4
5 print(texto.find("Robotics"))
6 print(texto.find("USP"))
7 print(texto.find("UFSCar"))
8
```

```
7
24
-1
```

- `find()`, procura da esquerda para a direita.
- `rfind()`, procura da direita para a esquerda, retornando a última ocorrência.

```
1 # "find" e "rfind"
2 texto = "banana"
3 print(texto.find("a")) # primeira ocorrência
4 print(texto.rfind("a")) # última ocorrência
```

```
1
5
```

```
1 # Contar o número de Ocorrências
2 texto = "banana"
3 print(texto.count("a"))
4
5 # o metodo count também aceita intervalos
6 print(texto.count("a", 0, 3)) # conta apenas entre os índices 0 e 2
```

```
3
1
```

Funções úteis de Strings

Além de buscar partes de um texto, o Python também oferece **várias funções nativas para manipular e transformar strings**. Essas funções são extremamente úteis no dia a dia, seja para **limpar, padronizar, dividir ou substituir** partes de um texto.

Elas nos permitem, por exemplo:

- Converter letras **maiúsculas e minúsculas** (`upper()`, `lower()`);
- **Formatar nomes e frases** com a inicial maiúscula (`capitalize()`, `title()`);
- **Substituir palavras** dentro de uma string (`replace()`);
- **Remover espaços indesejados** no início e fim (`strip()`);
- **Dividir textos** em partes (`split()`);
- **Unir listas** de palavras em um único texto (`join()`).

Essas funções são simples, mas muito poderosas — e dominá-las é essencial para trabalhar com qualquer tipo de dado textual, desde mensagens de usuários até grandes volumes de texto em projetos de análise de dados ou IA.

```
1 texto = "python é uma Linguagem de Programação"
2
3 print(texto.upper())      # Todas maiúsculas
4 print(texto.lower())     # Todas minúsculas
5 print(texto.capitalize()) # Somente primeira letra maiúscula
6 print(texto.title())     # Toda primeira letra da palavra maiúscula
```

```
PYTHON É UMA LINGUAGEM DE PROGRAMAÇÃO
python é uma linguagem de programação
Python é uma linguagem de programação
Python É Uma Linguagem De Programação
```

Em python as funções de busca que vimos anteriormente são case sensitive, ou seja, elas veem diferença entre letras maiúsculas e minúsculas. Por isso é comum quando fazemos busca em strings utilizar a função `lower`.

```
1 texto = "No laboratório de robótica da USP São Carlos têm robôs quadrupedes"
2 print("são carlos" in texto)
3 print("são carlos" in texto.lower())
```

```
False
True
```

Outro método utilizado para formatação é o `replace`. Ele permite substituir uma substring por outra.

```
1 frase = "Eu gosto de Java, aprendi muito."
2 nova_frase = frase.replace("Java", "Python")
3 print(nova_frase)
```

```
Eu gosto de Python, aprendi muito.
```

Também métodos de formatação muito utilizados para facilitar na busca e obtenção de informação dentro das strings são os métodos `strip()` e `split()`.

O `strip()` ele permite remover espaços e até outros caracteres indesejáveis na string. Vejamos os exemplos abaixo:

```
1 text = "  Laboratório de Robótica da USP São Carlos  "
2 print(text)
3 print(text.strip()) # removeu os espaços no começo e no final
```

```
Laboratório de Robótica da USP São Carlos
Laboratório de Robótica da USP São Carlos
```

```
1 text = "-----Laboratório de Robótica da USP São Carlos-----"
2 print(text)
3 print(text.strip("-")) # removeu os "-" no começo e no final
```

```
-----Laboratório de Robótica da USP São Carlos-----
Laboratório de Robótica da USP São Carlos
```

Ele também tem suas variações:

- `.lstrip()`, remove os da esquerda;
- `.rstrip()`, remove os da direita;

```
1 text = "-----Laboratório de Robótica da USP São Carlos-----"
2 print(text)
3 print(text.lstrip("-")) # removeu os "-" da esquerda
4 print(text.rstrip("-")) # removeu os "-" da direita
```

```
-----Laboratório de Robótica da USP São Carlos-----
Laboratório de Robótica da USP São Carlos-----
-----Laboratório de Robótica da USP São Carlos
```

O método `split()` serve para **dividir uma string** em várias partes menores, enquanto o método `join()` faz o **caminho inverso**, **juntando** várias partes de texto em uma única string novamente.

O `split()` separa um texto em partes menores, usando um **caractere separador** (por padrão, o espaço).

Por exemplo:

```
1 frase = "Robótica Móvel na USP"
2 palavras = frase.split() # Sem fornecer um caracter ele entende que o separador é o espaço
3 print(palavras)
```

```
['Robótica', 'Móvel', 'na', 'USP']
```

Você ainda não aprendeu sobre Listas, vamos ver esse assunto mais a fundo na próxima semana, mas, de forma simples, uma lista é uma estrutura que guarda vários valores juntos.

Então, o `split()` transforma uma única string em várias pequenas strings agrupadas, como se o Python guardasse cada palavra separadamente para podermos trabalhar com elas depois.

```
1 dados = "João,Pedro,Ana"
2 print(dados.split(",")) # Falamos que o separador é a ","
```

```
['João', 'Pedro', 'Ana']
```

O `join()` faz o oposto: ele pega várias partes de texto (como as criadas pelo `split()`) e junta tudo de volta em uma única string, usando um separador entre elas.

```
1 partes = ["Robótica", "Móvel", "na", "USP"]
2 frase_nova = " ".join(partes)
3 print(frase_nova)
```

```
Robótica Móvel na USP
```

```
1 print("-".join(partes))
2 print(", ".join(partes))
3
```